CSE4165 Web Programming — Final Exam Cheatsheet

1/2 · Boilerplates for the 3 questions (copy → edit)

Q1 · JavaScript — 2 files, 1 folder

index.html

```
<!DOCTYPE html>
<html>
<head><title>Title</title></head>
<body>
  <h2>Title</h2>
  <input type="number" id="a">
  <select id="op"><option>X</option></select>
  <input type="checkbox" id="chk"> Urban
  <button onclick="calc()">Submit</button>
  <h3 id="result"></h3>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
let total = 0, count = 0;   // globals = state
const goal = 2000;
function calc() {
  let v = parseFloat(
      document.getElementById("a").value);
  let op = document.getElementById("op").value;
  let urban =
      document.getElementById("chk").checked;
  if (isNaN(v) || v <= 0) {
    alert("Please enter a valid value");
    return;
  }
  total = total + v;
  count++;
  let msg = "";
  if (v <= 10)      { msg = "Low"; }
  else if (v <= 20) { msg = "Mid"; }
  else              { msg = "High"; }
  document.getElementById("result").innerHTML =
    "Total: " + total + "<br>" + msg;
  document.getElementById("a").value = "";
}
```

Recurring Q1 patterns

```
let list = [];  list.push(name);  count++;
let avg = total / count;
"Subjects: [" + list.join(", ") + "]"
if (count > 7 && total < goal) { warn }
el.disabled = true;     // stop more input
```

Q2 · PHP — form posts to process.php

index.html

```
<!DOCTYPE html>
<html>
<head><title>Title</title></head>
<body>
  <h2>Title</h2>
  <form action="process.php" method="POST">
    First:
    <input type="number" step="any" name="a">
    <br><br>
    Second:
    <input type="number" step="any" name="b">
    <br><br>
    <input type="submit" value="Calculate">
  </form>
</body>
</html>
```

process.php

```
<?php
$first = $_POST['a'];
$second = $_POST['b'];
function calculate($first, $second)
{
  $total = $first * $second;
  if ($total >= 500) {
    $level = "Excellent";
  }
  else if ($total >= 300) {
    $level = "Good";
  }
  else {
    $level = "Poor";
  }
  echo "<h2>Report</h2>";
  echo "Total: " . $total . "<br><br>";
  echo "Level: " . $level;
}
calculate($first, $second);
?>
```

Recurring Q2 formulas

```
// pizza / venue / movie / battery pattern
$units = ceil($need / $capacity);
$empty = ($units * $capacity) - $need;
$wasted = $empty * $price;
// compare: > Above, < Below, else exact
```

Q3 · PHP + MySQL — 3 files

db.php

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$database = "shop";
$conn = new mysqli($servername, $username,
                   $password, $database);
if ($conn->connect_error) {
  die("Failed: " . $conn->connect_error);
}
?>
```

index.php

```
<?php
include("db.php");
echo "<h2>Title</h2>";
$sql = "
SELECT Category, COUNT(*) AS Total
FROM items GROUP BY Category
";
$result = $conn->query($sql);
while($row = $result->fetch_assoc()) {
  echo "Category: " . $row['Category'] .
       " | Total: " . $row['Total'] . "<br>";
}
echo "<hr>";
$sql = "
UPDATE items SET Price = 0 WHERE ItemID = 1
";
$conn->query($sql);
echo "<h2>Updated</h2>";
$conn->close();
?>
```

necessary_SQL.txt

```
DROP DATABASE IF EXISTS shop;
CREATE DATABASE shop;
USE shop;
CREATE TABLE items (
  ItemID INT PRIMARY KEY,
  ItemName VARCHAR(30),
  Price DOUBLE,
  SaleDate DATE
);
INSERT INTO items VALUES
(1, 'Laptop', 50000, '2024-01-01'),
(2, 'Cox''s Pen', 20, '2024-01-03');
```

Run: XAMPP → start **Apache + MySQL** · folder in **C:\xampp\htdocs\exam** · open **localhost/exam/index.html** · phpMyAdmin → **SQL** tab → paste **necessary_SQL.txt** → Go · JS only: double-click index.html · re-run the SQL file to **reset data** (UPDATEs stack on every refresh).

CSE4165 Web Programming — Final Exam Cheatsheet

2/2 · Every SQL question type (one table) + JS / PHP syntax

Example table `items` — all SQL types below use it

ItemID	ItemName	Category	Price	Stock	Rating	SaleDate	Status
1	Laptop	Electronics	50000	5	4.8	2024-01-01	Sold
2	Mouse	Electronics	500	20	4.2	2024-01-02	Sold
3	Chair	Furniture	3000	8	4.5	2024-01-02	Pending
4	Desk	Furniture	7200	6	4.1	2024-01-03	Sold
5	Bottle	Accessories	150	40	3.9	2024-01-03	Returned
6	Pen	Accessories	20	100	4.6	2024-01-03	Returned

Number tags = exam papers where the type appeared (243 = Fall 2024).

1. Count rows per group (243 251 253 261)

```
SELECT Category, COUNT(*) AS Total
FROM items GROUP BY Category;
```

2. Total and average per group (242 252)

```
SELECT Category, SUM(Price) AS Revenue,
  AVG(Price) AS AvgPrice
FROM items GROUP BY Category;
```

3. Only groups with more than 1 row (HAVING) (253)

```
SELECT Status, COUNT(*) AS Total
FROM items GROUP BY Status
HAVING COUNT(*) > 1;
```

4. Top 2 days by revenue in January (242)

```
SELECT SaleDate, SUM(Price) AS Revenue
FROM items WHERE SaleDate
  BETWEEN '2024-01-01' AND '2024-01-31'
GROUP BY SaleDate
ORDER BY Revenue DESC LIMIT 2;
```

5. Rating above 4.5 OR name has Pen; sort (261)

```
SELECT ItemName, Category, Rating FROM items
WHERE Rating > 4.5 OR ItemName LIKE '%Pen%'
ORDER BY Rating DESC;
```

6. Price above 400 AND category Electronics (261)

```
SELECT ItemName, Price FROM items
WHERE Price > 400 AND Category = 'Electronics';
```

7. Total net worth (price x stock) (261)

```
SELECT SUM(Price * Stock) AS NetWorth FROM items;
```

8. Count + average + total per group; avg > 4.4 (261)

```
SELECT Category, COUNT(*) AS Items,
  AVG(Rating) AS AvgRating,
  SUM(Stock) AS TotalStock
FROM items GROUP BY Category
HAVING AVG(Rating) > 4.4
ORDER BY AvgRating DESC;
```

SQL question → answer (continued)

9. Value per group (price x stock) above 100000 (261)

```
SELECT Category, SUM(Price*Stock) AS Value
FROM items GROUP BY Category
HAVING SUM(Price*Stock) > 100000
ORDER BY Value DESC;
```

10. Groups sold on at least 2 different days (242)

```
SELECT Category, AVG(Stock) FROM items
GROUP BY Category
HAVING COUNT(DISTINCT SaleDate) >= 2;
```

11. 'Top Seller' if above its own category average (252)

```
SELECT i.ItemName, i.Price,
  CASE WHEN i.Price > c.AvgPrice
  THEN 'Top Seller' ELSE 'Regular Seller'
  END AS Label
FROM items i JOIN (
  SELECT Category, AVG(Price) AS AvgPrice
  FROM items GROUP BY Category
) c ON i.Category = c.Category;
```

12. Set a value if condition holds (243 251 252 253)

```
UPDATE items SET Status = 'Low'
WHERE Price < 1000 AND Status <> 'Returned';
```

13. Add bonus only if result stays within a cap (243 251)

```
UPDATE items SET Price = Price + 5000
WHERE Price > 3000 AND Price + 5000 <= 60000;
```

14. 10% discount (x0.90) or raise (x1.10) (252 253 261)

```
UPDATE items SET Price = Price * 0.90
WHERE Category = 'Furniture';  -- raise: *1.1
```

15. Update 2 columns at once (261)

```
UPDATE items SET Rating = Rating + 0.2,
  Price = Price * 1.10
WHERE Rating BETWEEN 4.0 AND 4.5
  AND Price > 0;
```

Gotchas

```
JS joins with +      PHP joins with .
PHP variables start with $
SQL: not equal <>  text in 'single quotes'
SQL: apostrophe in text -> 'Cox''s'
WHERE filters rows;  HAVING filters groups
Order: WHERE > GROUP BY > HAVING > ORDER BY
```

JavaScript syntax

```
parseInt("42")  parseFloat("3.5")  Number(x)
isNaN(x)    x.toFixed(2)   // string, 2 decimals
Math.floor(x)  ceil(x)  round(x)  abs(x)  sqrt(x)
Math.max(a,b)  Math.min(a,b)  Math.pow(a,2)  a%b
Math.floor(Math.random()*4501)+500   // 500-5000
n % 10 = last digit    Math.floor(n/10) = drop it
if (a > b) {..} else if (a == b) {..} else {..}
let s = x < 20 ? "bad" : x <= 30 ? "ok" : "good";
&&  ||  !    ===  !==    x >= 5 && x <= 10
s.length  s.trim()  s.toUpperCase()  s.split(",")
/[A-Z]/.test(s)  /[0-9]/.test(s)  /[!@#$%^&*]/
let a = [];  a.push(x);  a.length;  a.join(", ")
for (let i=0; i<a.length; i++) { sum += a[i]; }
while (n > 0) { n = Math.floor(n / 10); }
el.value  el.innerHTML  el.checked  el.disabled
alert("msg");      "Total: " + n + " BDT"
```

PHP syntax

```
$_POST['name']      isset($_POST['x'])
(int)$x  (float)$x  intval()  is_numeric($x)
ceil($a/$b)  floor()  round($x,2)  abs()  sqrt()
max($a,$b)  min($a,$b,$c)  pow($a,2)  $a % $b
(int)ceil($x)       // ceil gives a float
number_format($x, 2)       // 1,234.50
if () {..} else if () {..} else {..}
$s = $a > $b ? "A" : "B";   &&  ||  !  == === <>
strlen($s)   substr($s, -1)   // last character
strtoupper($s)  str_split($s)  explode(",", $s)
preg_match('/[A-Z]/', $s)        // 1 if found
$a = array(1,2); $a[] = 5; count($a); sort($a);
rsort($a)=desc  array_sum($a)  implode(", ",$a)
for ($i = 1; $i < $n; $i++) { ... }
while ($n > 0) { $n = (int)($n / 10); }
foreach ($a as $v) { echo $v; }
function f($a, $b) { return $a + $b; }
echo "A: " . $a . "<br>";      // . joins strings
$conn->query($sql)   $result->fetch_assoc()
$row['Col']  $result->num_rows  $conn->close()
$conn->real_escape_string($s)
```